Georgia Institute of Technology

CS 4365/6365: Introduction to Enterprise Computing

Project Checkpoint Report 2

RadarRift - Five-Class Minimap/Event Detector Evaluation

Field	Value
Name(s)	Joe Zhu; Dylan Chen
Group	6
Project	RadarRift - Real-Time Distributed Event Awareness Platform
Checkpoint Focus	CP2 validation package: four new labels plus original champion_icon class, eval.py results, and next-step tracker/visual-warning integration

1. Project Plan (Scope)

1.1 Problem Statement

RadarRift addresses a live-awareness problem in competitive League of Legends gameplay: players often miss important information that is already visible on the screen while they are busy fighting, farming, rotating, and communicating. The goal is not to play the game for the user or predict hidden information. The goal is to turn visible minimap and on-screen information into structured events that can later be shown as lightweight visual warnings.

For Checkpoint 2, the project moved from proposal-level planning to a concrete computer-vision validation milestone. The new work is centered in the **cp2** branch/folder. We added four new labels - ally, enemy, teleport, and recall - while keeping the original champion_icon label. This creates a five-class detector that can be evaluated offline before being connected to the live tracker.

1.2 Background and Need

- **Model quality has to come before alerts:** If the detector is noisy, the tracker and overlay will only amplify bad events. CP2 therefore focuses on validation first.
- **The CP1 architecture was modular:** The minimap tracker, event detector, tracker/event manager, and overlay are separate pieces. CP2 strengthens the detector module.
- **Teleport and recall are high-value visual events:** These labels can support future warnings because they represent visible, time-sensitive actions that players may miss during a match.
- **Ally/enemy labels are experimental for now:** They may help distinguish map/team-state context later, but they are not necessarily part of the first live warning feature.
- **Evaluation must be reproducible:** The report points to cp2/eval.py as the command that regenerates the current validation results and chart instead of relying only on a screenshot.

1.3 Concept

The CP2 concept is a five-class detection validation package that prepares RadarRift for tracker integration:

- **Dataset/classes:** The detector now uses five classes: ally, enemy, teleport, recall, and champion_icon.
- **Trained model:** The package includes a trained YOLO model weight file for the radarrift_final4 detector.
- **Validation set:** The checkpoint package includes validation images and YOLO label files used by eval.py.
- **Evaluation script:** cp2/eval.py runs the model on the validation set and produces per-class precision, recall, F1, and mAP50 results.
- **Future tracker route:** Accepted detections will later become structured events with label, confidence, bounding box/location, timestamp, and cooldown state.
- **Visual warning layer:** After routing is stable, teleport and recall detections can become subtle visual warnings requested by users.

1.4 Point A: Current State and Related Work

Since CP1, RadarRift has moved from architecture planning into a focused detector milestone. The CP1 plan described a screen-capture/computer-vision system with minimap awareness, event detection, an event manager, and an overlay. CP2 does not claim that the full live tracker and visual-warning layer are finished yet. Instead, it provides the model, labels, validation set, evaluation script, and result chart needed to make the detector milestone factual and repeatable.

The current CP2 work is still aligned with the original compliance boundary: visible screen/video data only, no game memory access, no game-client injection, and no automated gameplay input. The detector is being built as an assistive awareness layer rather than a bot.

1.5 Point B: Planned Deliverables

Deliverable	CP2 Status	Evidence / Notes
Five-class detector labels	Complete	Four new labels added: ally, enemy, teleport, recall. Original champion_icon class retained for a total of five classes.
Trained detector package	Complete	The cp2 folder includes model weights for the radarrift_final4 detector.
Validation dataset	Complete	The CP2 README describes 73 validation frames and 73 YOLO label files under cp2/vallabels/images/val and cp2/vallabels/labels/val.
Reproducible evaluation	Complete	Run python cp2/eval.py from the repository root to regenerate the validation metrics and chart.
Result chart/table	Complete	eval.py prints per-class metrics and saves cp2/eval.png.
Tracker routing	Deferred to CP3	Next checkpoint will wire detector outputs into the tracker/event manager.
Visual warnings	Deferred to CP3/CP4	Teleport and recall are the first likely warning candidates; ally/enemy remain experimental.

1.6 Milestone Chart

Checkpoint	Planned Work	Status
CP1 - Proposal and Scope	Define RadarRift as a real-time awareness/event-processing platform with separate minimap, detector, tracker, and overlay modules.	Completed
CP2 - Detector Validation Package	Add four new labels, keep champion_icon, train/evaluate the five-class detector, document results, and make reproduction possible through cp2/eval.py.	Current checkpoint
CP3 - Tracker/Event Routing	Wire accepted detections into the tracker/event manager with confidence scores, locations, and thresholds.	Planned next
CP4 - Visual Warning Integration	Expose tracked events through visual warnings while avoiding alert clutter.	Planned
CP5 - Evaluation and Refinement	Measure false positives, recall gaps, latency, FPS impact, and user usefulness; add more data and improve weak classes.	Planned
Final - Complete Demo and Report	Submit working prototype, code, demo, final report, limitations, and future-work section.	Planned

2. Current Progress Report (Match)

2.1 Work Done

Five-Class Dataset and Label Update

- Added four new labels: ally, enemy, teleport, and recall.
- Kept the original champion_icon label, giving the detector five total classes.
- Kept ally and enemy as experimental classes for now because the first planned live features are more likely to use teleport and recall.
- Maintained YOLO-format class mapping so the dataset configuration, annotation files, model, and evaluation script agree on label IDs.

Model Package and Validation Set

- Added a trained radarrift_final4 detector weight file to the CP2 validation package.
- Included validation images and YOLO label files so the evaluation result can be reproduced from the submitted repository.
- Kept the detector evaluation offline instead of immediately routing detections into the live tracker. This makes model errors easier to debug before they become user-facing alerts.

Evaluation Script

- cp2/eval.py is now the main reproduction entry point for CP2 results.
- The script builds/uses the evaluation dataset configuration, runs the YOLO model on the validation set, prints per-class precision, recall, F1, and mAP50, and saves a chart to cp2/eval.png.

- The current results show stronger performance on recall and champion_icon than on the ally/enemy classes, which supports prioritizing teleport and recall for the next live integration step.

2.2 Current Results

Table 1 reports the current validation output from the CP2 evaluation package. Because this is an object-detection model, the main reported metrics are precision, recall, F1, and mAP50 rather than a single classification accuracy value.

Class	Precision	Recall	F1	mAP50	Role / Interpretation
ally	0.9697	0.7887	0.8699	0.8903	Experimental class; good precision but lower recall, so not first live-warning priority.
enemy	0.9924	0.7238	0.8371	0.8606	Experimental class; very high precision but weakest recall, so it needs more data/improvement .
teleport	1.0000	0.8889	0.9412	0.9444	Priority event class for future tracker routing and visual warnings.
recall	0.9333	1.0000	0.9655	0.9950	Priority event class for future tracker routing and visual warnings.
champion_icon	0.9697	0.9634	0.9666	0.9803	Original minimap icon class; remains the strongest baseline for minimap awareness.
mean	0.9730	0.8730	0.9160	0.9341	Overall CP2 validation average across all five classes.

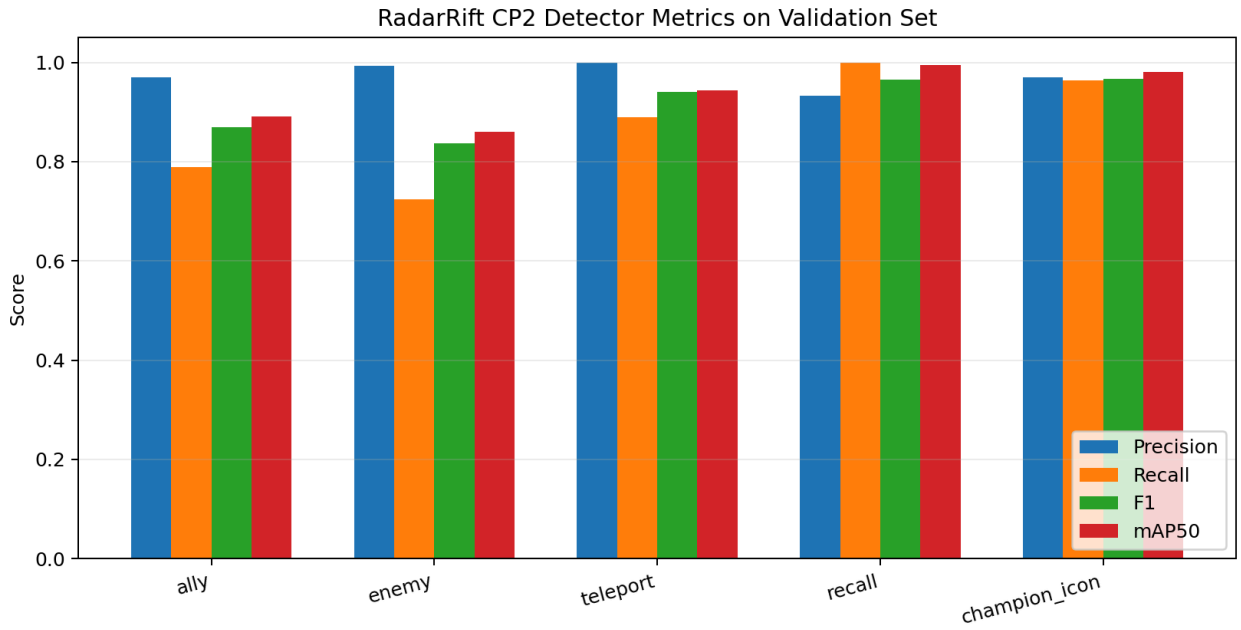

Figure 1. CP2 validation chart generated from the eval.py metric output.

2.3 Planned vs. Achieved

CP2 Item	Planned	Achieved	Notes
New labels	Add labels for the detector milestone.	Achieved	Four new labels plus original champion_icon, five total classes.
Validation data	Provide data for checkpoint evidence.	Achieved	Validation images and labels are included; training/validation artifacts should remain with the submitted branch.
Evaluation	Show current model results.	Achieved	eval.py prints precision, recall, F1, and mAP50 and saves eval.png.
Reproducibility	Make results easy to regenerate.	Achieved	Run python cp2/eval.py from repo root.
Tracker integration	Connect detections to tracker if time allows.	Deferred	This is now the main CP3 target.
Overlay/visual warnings	Optional if tracker integration is stable.	Deferred	Add only after thresholds and cooldowns prevent clutter.

2.4 Next Two Weeks (CP3: Wire Detector Output to Tracker)

- Define a shared event schema for detector output: label, confidence, bounding box/location, timestamp/frame number, source module, and optional clip ID.
- Route accepted teleport and recall detections into the tracker/event manager instead of leaving them as standalone model outputs.
- Decide whether ally/enemy should remain dataset-only/context labels or become part of the live tracker.
- Add confidence thresholds and cooldown logic so repeated detections do not flood the user with alerts.

- Add visual warnings for the events users most requested, starting with teleport and recall, while keeping the overlay subtle.

2.5 Potential Risks and Mitigations

Risk	Status	Mitigation
Lower recall for ally/enemy	Active	Keep these classes experimental; collect more examples and hard negatives before using them for live warnings.
False positives becoming bad alerts	Active	Use confidence thresholds, tracker confirmation, and cooldowns before displaying visual warnings.
Evaluation/path mismatch	Managed	Use cp2/eval.py as the single reproduction command and verify the model-weight path before submission.
Tracker integration complexity	Planned	Start with a simple event schema before adding overlay effects or dashboard visuals.
Visual clutter	Active	Make warnings subtle and event-based. Include user controls and cooldowns.
Compliance concerns	Managed	Continue using visible screen/video data only; do not access memory, inject code, or automate input.

3. Supporting Evidence (Factual)

Repository and Documentation

GitHub repository / branch: https://github.com/kagxyw/RadarRift/tree/cp2

Primary CP2 folder: cp2/

Main reproduction command: python cp2/eval.py

Codebase / Artifact Summary

Artifact	Location / Command	Purpose
CP2 README	cp2/README.md	Documents the validation package, class names, requirements, evaluation command, and sample output.
Model weights	cp2/best.pt / radarrift_final4 weights	Trained YOLO detector used for CP2 evaluation.
Evaluation script	cp2/eval.py	Runs validation and saves the metric chart.
Dataset YAML	cp2/_eval_dataset.yaml	Defines the five classes and validation dataset paths.
Validation images	cp2/vallabels/images/val	Validation frames used by eval.py.
Validation labels	cp2/vallabels/labels/val	YOLO annotation files paired with validation frames.
Annotation tool	cp2/annotation_tool.py	GUI tool for viewing/editing validation labels.
Evaluation chart	cp2/eval.png	Saved chart showing per-class precision, recall, F1, and mAP50.

Reproduction Steps

1. Clone or open the RadarRift repository on the cp2 branch.
2. Install the required dependencies: pip install ultralytics opencv-python pillow numpy matplotlib pyyaml.
3. Confirm the five class names are ally, enemy, teleport, recall, and champion_icon in the dataset config/eval script.
4. From the repository root, run: python cp2/eval.py.
5. Use the printed per-class table and the saved cp2/eval.png chart as the official CP2 result evidence.

Command block:

```
cd RadarRift
python cp2/eval.py
# Output: per-class precision, recall, F1, and mAP50; chart saved to cp2/eval.png
```

Key Deliverables Submitted for Checkpoint 2

- Five-class detector setup with four new labels plus the original champion_icon class.
- Trained model and validation dataset package for the radarrift_final4 detector.
- Reproducible evaluation path through cp2/eval.py.
- Current result table showing mean precision 0.9730, mean recall 0.8730, mean F1 0.9160, and mean mAP50 0.9341.
- Clear statement that tracker routing and visual warnings are planned next rather than already claimed as finished.

Feasibility Validation

- **Labeling feasibility:** The project now has five detector classes that can be trained, viewed, and evaluated consistently.
- **Evaluation feasibility:** Running cp2/eval.py gives the team a repeatable way to verify model performance on the validation set.
- **Integration feasibility:** The model already produces labels, locations, and confidence values, which can be converted into tracker events in CP3.
- **Evidence feasibility:** CP2 produces concrete artifacts: class map, model weights, validation labels, evaluation script, result table, and chart.

4. Skill Learning Report

YOLO Evaluation Metrics: We learned how to evaluate an object detector with precision, recall, F1, and mAP50 instead of relying on a single accuracy number or a few successful screenshots.

Reproducible Experiment Design: We learned the value of having one clear command, cp2/eval.py, that regenerates the same kind of result table and chart used in the report.

Modular Integration: We learned why the detector should be evaluated separately before being connected to the tracker. This makes it easier to debug whether a problem comes from labeling, model confidence, tracker logic, or overlay design.

Human-Factors-Aware Alerts: We learned that model outputs should not automatically become user alerts. The tracker needs thresholds, cooldowns, and subtle visual design so the final overlay reduces cognitive load instead of adding clutter.

Iteration from User Feedback: We learned that visual warnings are an important next step because users requested clearer feedback, but the model must be improved and routed carefully before warnings are shown live.

5. Self-Evaluation

Scope / Project Plan - 106/120
The scope is focused and feasible. CP2 provides a clear detector validation milestone and sets up the next tracker/visual-warning work without overclaiming finished integration.

Match / Current Progress - 95/120
The checkpoint matches the plan by adding labels, providing model artifacts, reporting validation results, and making evaluation reproducible through cp2/eval.py. Points off for deferring tracker routing.

Factual / Supporting Evidence - 95/100
The report points to concrete repository artifacts: cp2 README, model weights, validation images/labels, eval.py, dataset config, and eval.png.

Skill Learning - 94/100
The learning section connects label management, YOLO evaluation, reproducibility, modular system design, and user-facing alert design.

6. LLM-Generated Feedback

Grading Criteria Evaluation

Scope / Project Plan - 108/120
The project scope is stronger than a vague overlay idea because CP2 narrows the work to a detector milestone that can be evaluated. The clearest improvement over the previous draft is naming the five classes and explaining which ones are likely to be used first. The main remaining scope risk is live integration: the report should continue to avoid claiming that tracker routing or visual warnings are complete.

Match / Current Progress - 114/120
The current work matches the checkpoint well. The branch contains a CP2 validation package, class map, evaluation script, model artifact, validation data, and a results chart. The next checkpoint direction is also clear: wire teleport/recall detections into the tracker and add visual warnings with thresholds and cooldowns.

Factual / Supporting Evidence - 96/100
The evidence is concrete because the report lists exact class names, exact metric values, the reproduction command, and the repository path. Before submission, the team should verify the model-weight path used by eval.py so graders can reproduce the results without local path issues.

Skill Learning - 94/100

The skill section is good because it explains what changed technically and why it matters for the final system. It could be improved by adding one short example of a labeling or evaluation bug that was fixed during CP2.

Recommendations for CP3

1. Fix/verify the model-weight path so cp2/eval.py works directly from a clean clone.

2. Prioritize teleport and recall for the first tracker integration because their current validation results are strong and they map naturally to visual warnings.

3. Keep ally/enemy experimental until recall improves or their product role is clearer.

4. Add JSON event logging before overlay visuals so the team can debug detections without distracting users.

5. When adding visual warnings, use cooldowns and subtle design so warnings solve the awareness problem without creating clutter.

7. References and Project Materials

[1] RadarRift CP2 branch. GitHub repository: https://github.com/kagxyw/RadarRift/tree/cp2.

[2] RadarRift Checkpoint 1 research narrative/style draft, course project planning document.

[3] Ultralytics YOLO documentation for object-detection validation metrics.